

Scripting-Based 3D Geometry Modelling of Hairpin Windings in EV Traction Motors: Design for Manufacturing

Wenting Wang
Power Electronics and Machines Control
Research Institute
University of Nottingham
Nottingham, U.K.
wenting.wang@nottingham.ac.uk

Tianjie Zou
Power Electronics and Machines Control
Research Institute
University of Nottingham
Nottingham, U.K.
tianjie.zou@nottingham.ac.uk

Hailin Huang
Power Electronics and Machines Control
Research Institute
University of Nottingham
Nottingham, U.K.
hailin.huang@nottingham.ac.uk

Hua Li
Power Electronics and Machines Control
Research Institute
University of Nottingham
Nottingham, U.K.
hua.li@nottingham.ac.uk

Mingjie Wang
Power Electronics and Machines Control
Research Institute
University of Nottingham
Nottingham, U.K.
mingjie.wang@nottingham.ac.uk

Xiang Ren
Power Electronics and Machines Control
Research Institute
University of Nottingham
Nottingham, U.K.
xiang.ren@nottingham.ac.uk

Hadish Habte Tesfamikael
Power Electronics and Machines Control
Research Institute
University of Nottingham
Nottingham, U.K.
hadish.tesfamikael@nottingham.ac.uk

David Gerada
Power Electronics and Machines Control
Research Institute
University of Nottingham
Nottingham, U.K.
david.gerada@nottingham.ac.uk

Chris Gerada
Power Electronics and Machines Control
Research Institute
University of Nottingham
Nottingham, U.K.
chris.gerada@nottingham.ac.uk

Abstract—This paper presents a pure scripting-based and time-efficient 3D geometric modelling tool of hairpin windings for electrical vehicle traction motors. Developed with a “design for manufacturing” motivation, it enables optimizing winding design with both electrical and mechanical aspects of hairpin conductors considered. Without dependence on commercial CAD software, the tool can rapidly generate high-fidelity 3D models of windings with both welding and insertion end, based solely on winding layouts and geometric parameters of the stator. The modelling process is fully parameterized, enabling high flexibility for design iteration and manufacturing adaptation. Through a case study on a 48-slot, 8-pole, 6-layer hairpin winding stator, typical challenges regarding physical interference and manufacturing feasibility arising from pin arrangements are highlighted. To address these issues, tailored geometry tuning strategies are developed. Combined with these strategies, the tool integrates a partitioned checking and auto-tuning algorithm that can rapidly detect and resolve geometric interferences, ultimately producing collision-free and manufacturable 3D hairpin winding models.

Keywords—3D model, electric vehicles, end winding, hairpin windings, scripting.

I. INTRODUCTION

In recent years, hairpin windings have emerged as a revolutionary solution for boosting power density and efficiency of electric vehicles (EV) traction motors [1]. With needs of massive computer numerical control (CNC) manipulation manufacturing, they inherently feature “design for manufacturing”. Although the multi-layer configuration allows for a wide variety of theoretical winding layouts, the flexibility is in fact significantly constrained by manufacturability and high

risk of end-winding interference [2]–[4]. It makes the 3D geometry of hairpin conductors a critical factor at winding development stage, based on which industry R&D are competing against each other in terms of accelerating development cycle and reducing end winding size.

At present, commercial software tools can automatically construct 3D hairpin winding models with end connections based on user-defined parameters [5]. However, due to their rigid pin geometry, these tools are poorly suited for complex or non-standard layouts and often requires extensive manual adjustments just to eliminate interferences. Consequently, using these tools to explore optimal winding designs is highly impractical. Furthermore, this limitation leads the use of fixed end-winding resistance and leakage inductance values in 2D FEA analysis, which inevitably compromises the reliability of early-stage design iteration [6]. Additionally, the feasible model in the simulation may still fall short of manufacturability, demands substantial manual redesign before fabrication, exacerbating the simulation-to-reality deviation. These limitations collectively motivate the development of a more flexible and automated method that can rapidly generate high fidelity 3D hairpin winding models.

In [7], an automated model for generating production-oriented 3D insertion end models was introduced. The pin head geometry is defined using a smooth, piecewise cubic-spline centerline, as Fig. 1(a) shows. A rectangular cross-section is then swept along the spline to generate a 3D mesh for both visualization and collision checking. These control points on splines of all pins can serve as variables in a global optimization

for minimizing wire length under manufacturing constraints such as minimal bending radii, spatial clearances, etc.. However, this approach still relies on a reasonably good initial design to obtain the effective control points. Building on a similar segmentation concept, another construction approach was proposed, in which a partitioned ‘wedge’ unit is identified first, as Fig. 1(b) illustrates [8]. This unit encapsulates a set of segments that can form a single pin head by lofting. Both models embed collision-checking function and geometric parameters of control points or segments are iteratively adjusted to yield a collision-free model. However, a shared limitation is that they are applicable only to winding models with a single, uniform hairpin shape. Built in Grasshopper, a design tool that can generate the full 3D CAD models was developed driven by the full winding layout and stator geometry [9]. It constructs a centerline similar to [7], but derives the key geometric points directly from the start and end positions of each pin based on the winding layout. Manufacturing constraints like springback [10] are also incorporated during the process. The tool also supports real-time adjustment but relies on Rhinoceros 3D’s visual scripting environment. Nevertheless, the tool currently lacks ability to identify complex interconnections such as jumpers and does not incorporate automatic collision detection. These regions, however, are where interference is most likely to occur, as will be discussed later. Consequently, manual intervention are necessary to ensure a valid, collision-free model.

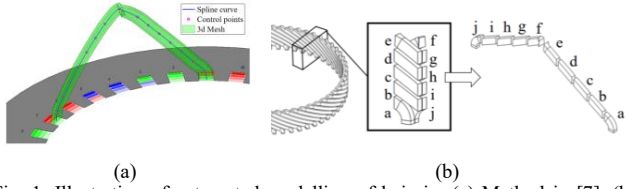


Fig. 1. Illustration of automated modelling of hairpin. (a) Method in [7]. (b) Method in [8].

In this paper, the scripting-based tool built in MATLAB is developed for fast high-fidelity 3D hairpin winding modelling without reliance on commercial CAD software or manual intervention. The proposed tool integrates layout-driven 3D geometry generation, partitioned interference checking, and parameter auto-tuning to deliver interference-free 3D winding models including both insertion and welding-ends in approximately one minute. With this parametric framework, physical interference are identified and manufacturing feasibility are discussed. By incorporating tailored geometry tuning strategies, the tool demonstrates robust capability in resolving collisions in winding models with complex interconnections.

II. SCRIPTING-BASED TOOL FOR AUTOMATED DESIGN AND MODELLING OF HAIRPIN WINDINGS

This section elaborates the modelling concept for 3-D hairpin windings based only on winding layout and stator & conductor parameters and explains how it is implemented in the script. A 48-slot, 8-pole, 6-layer stator is taken as an example, whose one-phase winding layout is illustrated in Fig. 2. As Fig. 2 shows, all of pins can be classified into three types based on their pitch span: short-pitched (5-slot), long-pitched (7-slot), and full-pitched (6-slot). In this layout, the short- and long-pitched pins locate in the innermost and outermost layers. For both of

these groups, the two legs of each pin locate in the same layer. By contrast, the full-pitched pins occupy the intermediate layers, having their two legs positioned in adjacent layers.

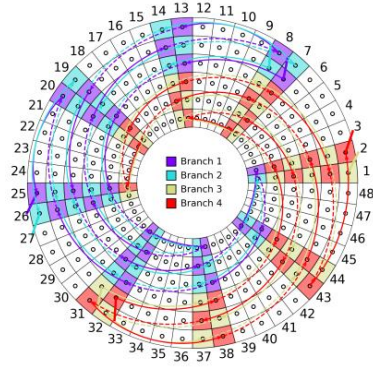


Fig. 2. Hairpin circular winding layout view of a 48-slot, 8-pole, 6-layer design. Solid line: insertion side; Dot line: welding side; Numbers: slot numbers. The inner to outer side corresponds to the path from the rotor to the stator yoke.

TABLE I. GIVEN MACHINE PARAMETERS

Parameter	Value
Radius of center at the innermost layer, r_6	77.225 mm
Radius of center at the outermost layer, r_1	90.475 mm
Gap between two conductors, g_c	0.25mm
Extension length at one end side, l_{ext}	4 mm
Width of conductor, w_s	5 mm
Thickness of conductor, t_s	2 mm
Stack length, l_{st}	120 mm

From the view of mechanical structure, each pin comprise an insertion end, a straight section, and a welding end section, as shown in Fig. 4(a). With the given layout in Fig. 2 and key parameters in TABLE I and Fig. 3, the 3D coordinates of the eight vertices that define the straight section of every pin can be obtained directly. Then, the focus of this study is therefore the geometric construction of the twisted welding end and the complex insertion end.

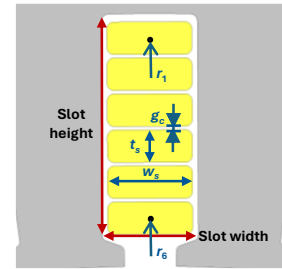


Fig. 3. Illustration of stator and conductor parameters.

To link the two straight legs, the center-line trajectory of insertion end is first defined by three successive primitives and is then smoothed into a single curve with helical and spline segments, as shown in Fig. 4(a) and (b). The three primitives are:

(i) a leading circular arc (curve B) that starts from the centre of the first leg’s top face,

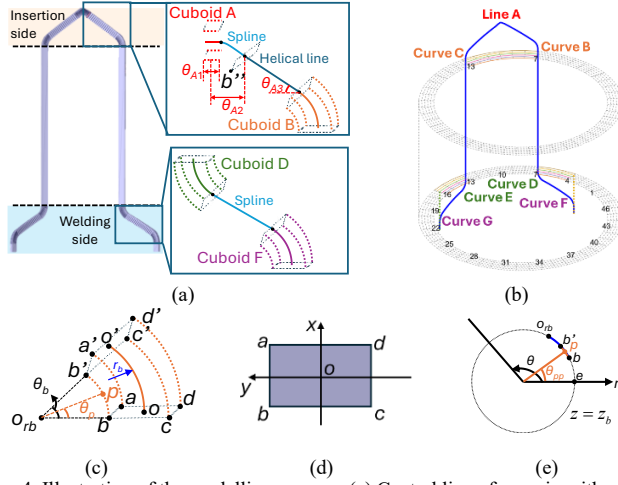


Fig. 4. Illustration of the modelling process. (a) Central line of one pin with two legs in slot 7, layer 2 and slot 13, layer 3, respectively. (b) Key cuboids in one pin model. (c) Illustration of the bent cuboid B. (d) Cross-section of the pin in its local coordinate. (e) Projections on $z=z_b$ plane.

(ii) a short apex straight line (line A) at the top, and

(iii) a trailing circular arc (curve C) that arrives at the centre of the second leg's top face.

Likewise, the cuboids of welding end can be constructed. Curves D and F (E and G) define its entry and exit paths, and a spline line between them blends the two.

With the centre-line trajectory fixed, the constant cross-section rule allows the proposed script directly trace the four corner paths and then form the solids. For insertion side, cuboid A and the bent cuboids B and C (pin shoulder) are built first, then spline- and helix-based prisms bridge the gaps to form a continuous 3D insertion end. It is important to note that, no sweep operation is used in our model, i.e. every solid is constructed purely through edge definition.

In the following, the modelling concept of the insertion end of one pin is presented in detail.

For the bent cuboid B in Fig. 4(a), with parameters in TABLE I, the 3D coordinates of the center of the cross section $\square abcd$ at axial position $z=z_b=l_{st}/2+l_{ext}$ (top face of the straight leg) can be easily obtained in the cylindrical coordinate system as:

$$o = (r_l, \theta_{nsb}, z_b), \quad (1)$$

where r_l is the radius position of the center o at layer l and θ_{nsb} is the angular position of center o at slot n . The bending of cuboid B is governed by a centerline arc oo' characterized by bending radius r_b and bend angle θ_{in} . In this study, the two parameters are manually set as:

$$r_b = 2w_s, \quad (2)$$

$$\theta_b = \frac{\pi}{2} - \theta_{A3} = \frac{\pi}{2} - \arctan \frac{h_{end}}{l_{csp}/2}. \quad (3)$$

where h_{end} is the manually defined axial length of the insertion end and l_{csp} is the coil span length. Then, offsetting this guide arc radially by $\pm t_s/2$ gives four concentric arcs aa' , bb' , cc' , and dd' , maintaining constant cross-section dimensions. Taking arc

bb' as an example, according to the relative position between vertex b and center o in Fig. 4(d), its bend radius is:

$$L_{bo_{rb}} = r_b - w_s / 2, \quad (4)$$

where o_{rb} is its arc center. From the projection on the $z=z_b$ plane in Fig. 4(e), the cylindrical coordinates of o_{rb} are:

$$o_{rb} = (r_l - t_s / 2, \theta_{o_{rb}}, z_b), \quad (5)$$

$$\theta_{o_{rb}} = \frac{L_{eo_{rb}}}{r_l - t_s / 2} = \theta_{ns} + \frac{r_b}{r_l - t_s / 2} = \theta_{ns} + \frac{L_{bo_{rb}} + w_s / 2}{r_l - t_s / 2}. \quad (6)$$

From Fig. 4(e), it can be easily inferred that the radial position of any point p in arc bb' equals to $(r_l - t_s / 2)$. In terms of the axial position of any discrete point p , based on Fig. 4(c), it can be calculated by:

$$z_p = z_b + L_{bo_{rb}} \sin \theta_p = z_b + (r_b - w_s / 2) \sin \theta_p, \quad 0 \leq \theta_p \leq \theta_b \quad (7)$$

Its angular position can be determined from arc projections in Fig. 4(e):

$$\theta_{pp} = \frac{L_{eo_{rb}} - L_{bo_{rb}} \cos \theta_p}{r_l - t_s / 2}, \quad (8)$$

where the length of arc eo_{rb} can be obtained from (5) and (6).

Applying above steps to obtain arcs aa' , bb' , cc' , it yields all edges that enclose bent cuboid B. In the numerical implementation, each arc is discretized into multiple points to ensure smooth geometric representation.

After the four arcs are established, each arc is extended into a helical edge curve tangent to its endpoint, as shown in Fig. 3(a). Along the helix, the radial position is fixed at $r \pm t_s / 2$. Then, to join the two corresponding helices from cuboids B and C, their termination points need to be defined and then connected through a cuboid A at the top region.

The script positions the cuboid A at the mid-radius and mid-azimuth of the two legs, occupying a fixed azimuth of $\theta_{A1}=1^\circ$ in Fig. 3(a). The axial position is defined by:

$$z = z_b + h_{end}. \quad (9)$$

Accordingly, the eight vertices of cuboid A follow directly.

The next is to determine the endpoint of each helix. The azimuthal gap between its endpoint and the centroid of cuboid A, denoted as θ_{A2} in Fig. 4(b) is set to be:

$$\theta_{A2} = 0.4\theta_{sp}, \quad (10)$$

where θ_{sp} is the slot pitch. In this way, the end point of the helix that originates at vertex of cuboid B can be obtained. For example, the coordinates of the end point b'' are:

$$b'' = \left[r_l - \frac{t_s}{2}, \theta_{b''}, \left(r_l - \frac{t_s}{2} \right) (\theta_{b''} - \theta_{b'}) \tan \theta_{A3} \right], \quad (11)$$

$$\theta_{b''} = \frac{\theta_{nsb} + \theta_{nsc}}{2} - \theta_{A2} + \frac{\frac{w_s}{2} \sin \theta_{A3}}{r_l - \frac{t_s}{2}}. \quad (12)$$

Finally, each helix endpoint is joined to cuboid A by a cubic-spline transition prism. The edge of prism is created in MATLAB with a single Hermite segment whose end-point derivatives are set equal to the tangents of the adjoining helical

and cuboid edges. This guarantees position and first-derivative continuity at both junctions. Together, the two bent cuboids, two helices, cuboid A, and the two prisms form a continuous 3D insertion end geometry.

Applying the same routine to every pin based on their location information from the winding layout, the script can automatically compute their edge trajectories, as highlighted in red in Fig. 5(a). The four edge trajectories of each pin are then resampled to ensure uniform point distribution. Then, adjacent sections will be triangulated slice-by-slice with MATLAB's 'triangulation' function. This results in each pin being composed of hundreds of interconnected triangular elements that precisely approximate its 3D geometry. At last, the resulting watertight mesh is exported to STL format via the built-in 'stlwrite' function in MATLAB, as exhibited in Fig. 5(b).

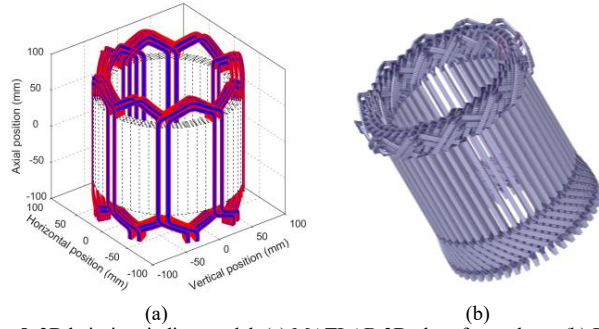


Fig. 5. 3D hairpin winding model. (a) MATLAB 3D plot of one phase. (b) Full model in STL format.

III. INTERFERENCE MITIGATION AND MANUFACTURABILITY CONSIDERATIONS

Using the developed script, critical interference issues at the insertion end are identified, and the manufacturability of the welding end are considered for the selected winding layout. Furthermore, comprehensive geometric methods to mitigate interference are proposed.

A. Interference between Adjacent Pins in Intermediate Layers

For two adjacent pins both with legs in adjacent intermediate layers, interference is likely to occur at the pin head. As Fig. 6(a) shows, if the user-defined h_{end} is too small, the two helices from opposite legs may overlap before reaching cuboid A. In this case, spline-based transition prism can protrude into the other pin and hence create collision.

It is straightforward that increasing h_{end} or/and additionally raising cuboid A can directly eliminate intersections, as Fig. 6(b) illustrates. However, this method inevitably results in longer conductor length, higher copper usage, and larger volume.

An alternative solution is to apply a mild twist at the pin head, i.e. rotate cuboid A by an appropriate angle, as depicted in Fig. 6(c). This rotation changes the orientation of the faces that anchor the spline transitions. The adjusted face normals deflect the initial tangents of the splines, causing them to peel away earlier from the neighbouring pin's helical prism. Compared with the previous solution, this method not only provides extra vertical and lateral clearance between the adjacent pins but also keeps the overall pin shape and length nearly unchanged.

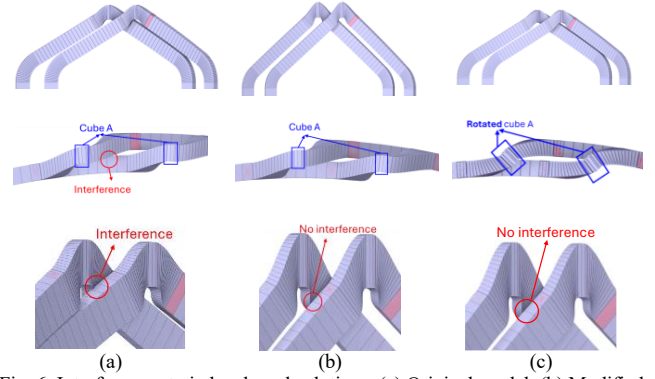


Fig. 6. Interference at pin heads and solutions. (a) Original model. (b) Modified model with increased height. (c) Modified model with rotated cuboid A.

B. Interference between Jumpers in Inner-/Outermost Layer

For short- and long-pitched jumpers of the same phase in the same layer, the interference would not occur as long as the h_{end} is sufficient. Nevertheless, jumpers belonging to different phases inevitably overlap if following the same modelling process in Section II. For example, as highlighted in Fig. 7, the leftward bend of a phase A jumper may interfere with the rightward bend of a neighbouring jumper of phase B in the innermost layer. This occurs because both pins attempt to occupy the same radial space when connecting to their respective cuboids A.

One solution is bending cuboids B (or C) of all jumpers slightly outward, as red arrows in Fig. 7 illustrate. In the model, this can be achieved by gradually shifting the radial position of the bending center when building cuboid B. To be specific, assuming N_b discrete points along curve bb' , the bending center of the k -th point is adjusted from (5) to:

$$o_{rb} = (r_l - t_s) / 2 + \frac{r_{br}}{N_b - 1} (k - 1), \theta_{o_{rb}}, z_b), \quad (13)$$

where r_{br} is a user-defined radial offset as illustrated in Fig. 8(a). As a result, the helix originating from cuboid B will start at a radially shifted position of $r_l \pm t_s + r_{br}$. Meanwhile, cuboid A will automatically reposition itself to the new midpoint at $r_l \pm t_s + r_{br} / 2$. From Fig. 8(b), it can be seen that the rightward helix of jumper of phase B now sits further out, making room for the leftward helix of jumper of phase A. Comparison between Fig. 7 and Fig. 8(c) clearly shows the elimination of the interference.

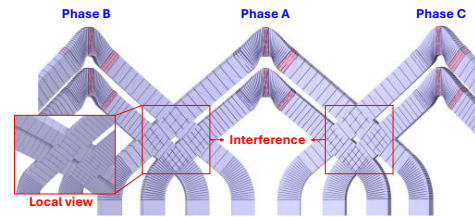


Fig. 7. Interference between the jumper pins.

It should be noted that this solution requires coordinated adaptation across all winding layers to prevent interference between layers, as Fig. 9 shows. It is worth noting that, pins in the outermost layer, which have identical layout to those in the innermost layer, require different radial offsets for cuboids B and C to prevent similar interference.

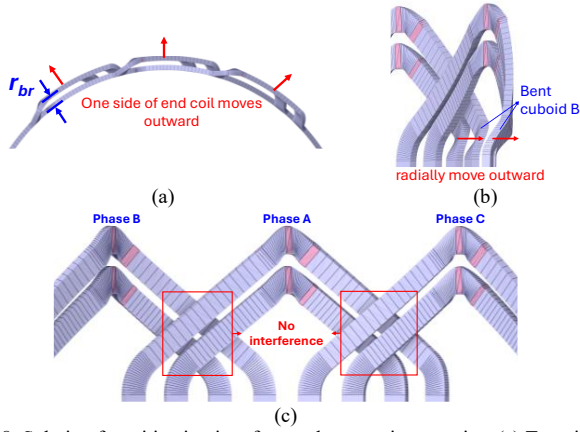


Fig. 8. Solution for mitigating interference between jumper pins. (a) Top view. (b) Perspective view. (c) Front view.

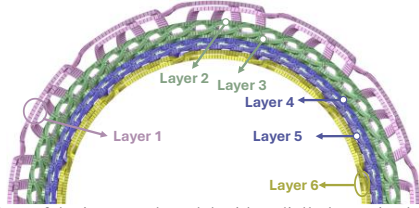


Fig. 9. Top view of the improved model with radially bent pin shoulders.

C. Manufacturability Consideration on Welding Side

Despite uniformity, the design of welding side needs to account for the reliability during the contacting process. In the original model, the cuboids of the welding end of the same pin are positioned at the same radial position, as Fig. 10(a) exhibits. It leads to an extremely narrow gap between welding joints, typically smaller than 1mm. However, such a limited spacing is insufficient for reliable programmed laser welding, which requires adequate clearance to enable accurate recognition of welding joints by the automated system. Moreover, a sufficient gap is significantly critical in mitigating the risk of short circuits caused by molten metal spatter during the welding process [11].

Therefore, to add an additional clearance, the radial position of the whole cuboid G (or F) at the end of the pin is shifted outward by $r_w=3\text{mm}$, as Fig. 10(b) shows. Given the transition prism is automatically constructed based on a spline curve fitted between the positions of cuboids D and G (or E and F), its shape will naturally bend to accommodate the modification.

IV. INTERFERENCE CHECK AND SELF-TUNING FRAMEWORK FOR MANUFACTURABLE HAIRPIN WINDING MODEL GENERATION

This section introduces interference-checking and auto-tuning framework embedded with the proposed mitigation solutions to generate a manufacturable 3D hairpin winding model. The framework incorporates a closed-loop that can rapidly detect specific interferences, and then auto-tune the region-specific parameter(s) within predefined suitable ranges to resolve collisions. By executing the check-and-tune loop in an inside-out, layer-by-layer order, the script ultimately delivers a fully non-interference 3D winding model.

Since each pin is represented by a triangular surface mesh, as long as interference state exists in any triangle pairs are found between the two pins, as Fig. 11(a) shows, the collision exists in

two pins. Therefore, for interference detection, a triangle intersection-based approach is implemented.

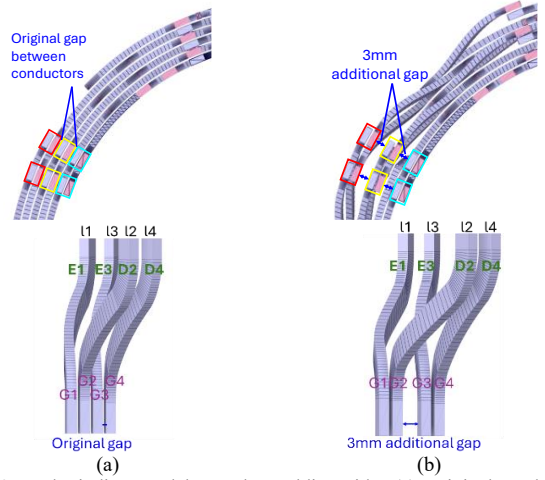


Fig. 10. End-winding models on the welding side. (a) Original model. (b) Modified model.

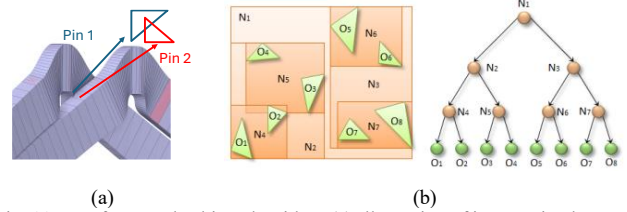


Fig. 11. Interference checking algorithm. (a) Illustration of intersection between two 3D triangles. (b) BVH collision checking algorithm [12].

For each candidate triangle pair from different pins, a rigorous intersection check is performed using the Separating Axis Theorem (SAT). The SAT states that two convex shapes are disjoint if and only if a separating axis exists where their projections do not overlap. For triangle-triangle intersection check, this requires checking 11 critical projection directions to definitively determine overlap. The algorithm also employs early termination by immediately concluding non-intersection when any separating axis is found, which accelerates the detection process. Only when all projection tests show overlap is a collision between two triangles and hence two pins confirmed.

While the SAT-based triangle intersection test provides rigorous collision detection, its direct application to the full model faces computational challenges. If the surface mesh of each pin comprises N (hundreds) triangular elements, $O(N^2)$ pairwise checks are needed between two pins. For a full model containing M pins, it requires $O(M^2 \cdot N^2)$ intersection checks.

To address this issue, two methods are adopted for acceleration. First, the Bounding Volume Hierarchy (BVH) structure recursively partitions the triangular meshes into spatially organized bounding volumes as illustrated in Fig. 11(b). Every non-overlapping boxes at each hierarchy is discarded so that only the few small boxes that might truly collide undergo exact SAT-based checks. As compared in TABLE II, the BVH-based method achieves orders-of-magnitude speedup compared to the all-pair checked method. For the case with 10 pins, the all-checked method needs 300s while the BVH-based method

completes in 8.5s, reducing over 95% computation time. The case with 48 pins further indicates BVH-based method becomes increasingly effective as the pin number grows.

TABLE II. COMPARISON OF CALCULATION TIME USING DIFFERENT INTERFERENCE CHECKING METHODS (UNIT: S)

Number of pins	All checked	BVH
3	20	1.4
6	100	3.5
10	300	8.5
48	...	190

Second, a partitioned checking strategy is implemented, which synergistically combines geometric symmetry, spatial partitioning, and layer-sequential checking. As illustrated in Fig. 12, since insertion end will not interfere with welding end due to their spatial segregation, two ends are separated and checked internally. Moreover, leveraging the inherent symmetry, the interference checking process is applied only to the smallest repeating unit which is sufficient to determine all necessary parameters for constructing the full model.

From Fig. 2, it can be seen that there are only two mechanical pin shapes in the innermost layer, the long-pitched and short-pitched pin pair belonging to the same phase lying side by side along the radial direction. This pair forms the minimal repeating unit of the entire pin pattern in this layer. Therefore, interference checking is first conducted on these two pins. Once interference is detected, the script immediately transitions into the tuning phase, where the end height (h_{end}) of the long-pitched pin is increased by Δh . Then, the new insertion end model is built and rechecked until no collision occurs. However, as discussed in Section III. B, additional interference may occur between pins of different phases. To address this, another check is performed

on four pins from two phases (Blue and purple pins). In this step, the bending radius (r_{br}) of cuboid B is iteratively adjusted until the interference is resolved. With both h_{end} and r_{br} properly determined, all pins in the innermost layer are guaranteed to be collision-free.

For pins in intermediate layers, three specific regions are checked for potential interference. First, interference between two adjacent pins of the same layer is examined. The twisting angle of cuboid A is adjusted to avoid pin head intersections as noted in Section III. A. Second, interference between pins of the innermost layer and the fifth layer is assessed. Specifically, their potential interference can be found by examining the two same-phase pins in the innermost layer and the six surrounding pins positioned behind them. This type of interference is mitigated by increasing r_{br} of both cuboids B and C of pins in the fourth/fifth layer. Third, to avoid interference between intermediate layers, one representative pin from the inner layer and six pins from the adjacent outer layer are selected for checking. For pins in the outermost layer, the structural configuration are identical to that of the innermost layer, and thus a similar checking process can be applied. However, an additional check is required to evaluate potential interference with the adjacent intermediate layer.

On the welding side, the process is significantly simplified due to the uniform geometry. Only interference between adjacent ends within the same layer needs to be checked.

Fig. 12 summarizes all the minimal units that require checking, along with their corresponding tuning strategies. Following this inner-to-outer checking sequence, a set of feasible pin parameters can be obtained. Then, for each layer, the minimal pin set is first constructed using the methodology in Section II, and the remaining pins are generated by angular replication. After triangulation, a STL file of the full 3D winding model can be output.

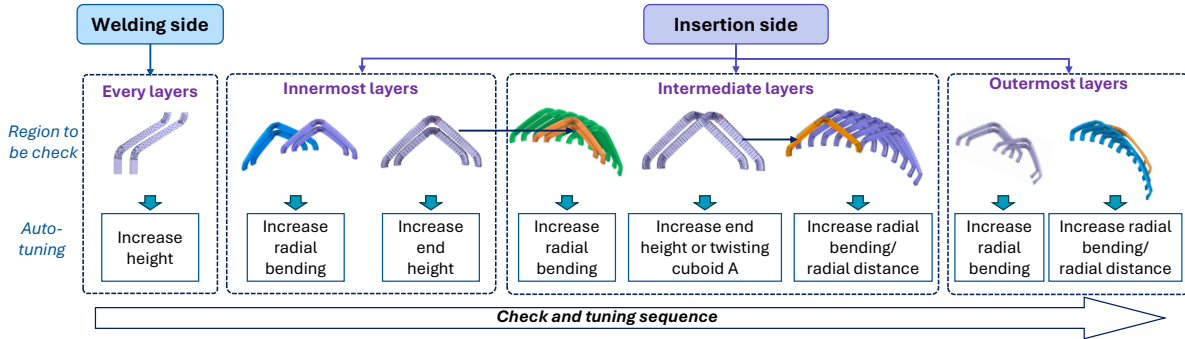


Fig. 12. Illustration of efficient interference check and auto-tuning strategy.

To validate the proposed method, it is implemented as a fully script-based tool in MATLAB 2025a. For the machine with parameters in TABLE I, the complete 3D model is generated in 18.33s. To further demonstrate the robustness and efficiency of the tool, two additional stators are selected as case studies, with their key parameters summarized in TABLE III. Fig. 13 presents the final models exported in STL format and visualized as 3D geometries. The corresponding modelling times, as listed in the table, show that the tool can generate high-fidelity, interference-free 3D hairpin winding models in approximately one minute.

TABLE III. KEY PARAMETERS AND MODELING TIME FOR CASE STUDIES

Case	r_6	r_1	w_s	t_s	g_c	r_w	Modelling time
1	90.625	77.09	4.678	2.507	0.2	3	64.2s
2	100.1	85	4	2.7	0.2	3	50.5s

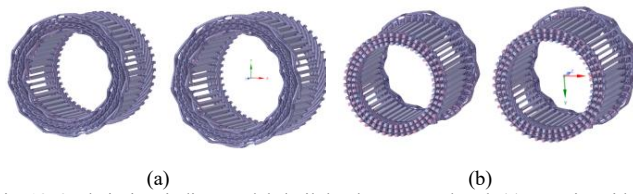


Fig. 13. 3D hairpin winding models built by the proposed tool. (a) Insertion side view. (b) Welding side view.

V. CONCLUSION

This paper develops a scripting-based and winding layout driven tool for the automated modelling of 3D hairpin windings for manufacture. The tool incorporates efficient interference check and self-tuning capabilities, which show strong potential for design optimization and comparison of different winding geometries in terms of both manufacturability and electromagnetic performance at the early design stage.

ACKNOWLEDGMENT

This work was supported by Innovate UK Project Monumo's 3D Generative-AI Tool (3D-GAIT) for Rapid and Manufacturable Engine Motor Designs under grant agreement number 10102233.

REFERENCES

- [1] H. Huang et al., "Continuous Hairpin Winding Technology for Electric Machines Enabling Net Zero Transportation: A Comprehensive Review," *IEEE Open J. Veh. Technol.*, vol. 6, pp. 1481-1500, 2025.
- [2] T. Zou et al., "A Comprehensive Design Guideline of Hairpin Windings for High Power Density Electric Vehicle Traction Motors," *IEEE Trans. Transport. Electrification*, vol. 8, no. 3, pp. 3578-3593, Sept. 2022.
- [3] L. P. Eschenmann, S. Aufderheide, and B. Ponick, "Method for efficient generation of preferable hairpin winding diagrams," in *Proc. of IECON Annual Conf. of IEEE Ind. Electron. Soc.*, Singapore, Oct. 2023, pp. 1-6.
- [4] W. Zhang, Z. Wu and W. Hua, "Design of n-Layer Flat Wire Winding in Permanent Magnet Machine for Electric Vehicles," *IEEE Trans. Transport. Electrification*, vol. 11, no. 3, pp. 7798-7810, June 2025.
- [5] S. Paul and B. Cebulski, "Revolutionizing Motor Design: Enhancing Efficiency with 3D End-Winding Simulations in JMAG-Designer," PowerSys, [Online]. Available: <https://global.powersys-solutions.com/learn/revolutionizing-motor-design-enhancing-efficiency-with-3d-end-winding-simulations-in-jmag-designer/>
- [6] M. Silberberger, D. P. Morisco, H. Rapp and A. Möckel, "Calculation of End-Winding Leakage Inductance for Hairpin Winding High Power Density Traction Machines Using the PEEC Method," in *Proc. of IEEE Int. Elect. Mach. Drives Conf.*, Hartford, CT, USA, 2021, pp. 1-7.
- [7] L. Hausmann, F. Wirth, and J. Fleischer, "Opportunities of Model-Based Production-Oriented Design of Stators with Hairpin Winding," in *Proc. of IEEE Int. Elect. Drives Prod. Conf.*, Ludwigsburg, Germany, Dec. 2020, pp. 1-8.
- [8] H. Mogg-Walls, A. Nassehi, M. Goudswaard, and N. Simpson, "Development of a Computational Design Tool for the Automatic Routing of Hairpin End-Windings," in *Proc. of Int. Conf. Power Electron. Mach. Drives*, Nottingham, UK, 2024, pp. 539-546.
- [9] C. Putz, J. H. Schleifenbaum, S. Ziegler, S. Reich, and A. Gruttke, "Design Automation: Digital Design Configurators of Hairpin Windings," in *Proc. of Int. Expert Forum Electric Vehicle Drives E-Mobility*, Schweinfurt, Germany, Sept. 27-28, 2023, pp. 187-194.
- [10] F. Wirth, J. Fleischer, J. Gerner, and L. Hausmann, "Closed-Loop Process Control for Sequential Tool-Bound Bending of Hairpin Coils," in *Proc. of IEEE Int. Electric Drives Prod. Conf.*, Regensburg, Germany, Nov. 2023, pp. 1-8.
- [11] M. Baader, A. Mayr, T. Raffin, J. Selzam, A. Kuhl, and J. Franke, "Potentials of Optical Coherence Tomography for Process Monitoring in Laser Welding of Hairpin Windings," in *Proc. of IEEE Int. Elect. Drives Prod. Conf.*, Erlangen, Germany, Dec. 2021, pp. 1-10.
- [12] T. Karras, "Thinking Parallel, Part II: Tree Traversal on the GPU," NVIDIA Developer Blog, Nov. 2012. [Online]. Available: <https://developer.nvidia.com/blog/thinking-parallel-part-ii-tree-traversal-gpu/>